A CASE-STUDY BASED REPORT

ON

Soil Quality Classification Using Decision Tree and Random Forest Machine Learning Models

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

P.NAVEEN **:2520030518**

J.ADVAITH **:2520030524**

P.SHASHE PREETHAM **:2520030534**

Under the Esteemed guidance of

Mr. G. Ravindra Babu

Professor, Department of CSE
Team 19 • Section 6 | Course Code: 25SC2107E
K L Deemed to be University, Bachupally
Hyderabad – 500090, Telangana, India



ABSTRACT

This case study presents a machine-learning approach for classifying soil quality into Low, Medium, and High categories using six measured or supplied soil attributes: pH, nitrogen, phosphorus, potassium, organic matter, and moisture. The project compares a Decision Tree classifier, which offers relatively transparent decision rules, with a Random Forest ensemble that combines predictions from many trees. The supplied project folder contains a 1,500-row CSV dataset, saved model artifacts, evaluation metrics, and a dashboard for entering soil parameters and viewing predictions and model analytics. The saved metrics report a test accuracy of 63.67% for the Decision Tree and 74.00% for the Random Forest; their reported five-fold cross-validation means are 67.27% and 75.73%, respectively. These results indicate that the Random Forest performed better in the recorded evaluation. However, the available CSV uses the target-column name Soil_Type while the current training script expects Soil_Quality, so the complete pipeline should be reconciled and rerun before treating the reported results as a fully reproducible final evaluation. The system is intended as a decision-support prototype, not a replacement for laboratory soil testing or expert agronomic advice.

1. INTRODUCTION

1.1 Background

Soil condition influences crop growth, nutrient availability, water retention, and farm productivity. Soil assessments commonly consider chemical and physical indicators such as pH, nutrient levels, organic matter, and moisture. Interpreting several indicators together can be time-consuming, especially when users need a quick preliminary view of soil condition. Machine learning can learn patterns from labelled examples and map a set of soil measurements to a class such as Low, Medium, or High.

This project implements a comparative classification workflow using two supervised, tree-based algorithms. The project files include a data-generation script, a model-training script, serialized model files, metrics in JSON format, plots, and a web-style interface. The report describes what is present in the supplied project rather than assuming that every component has been independently rerun.

1.2 Motivation for the Project

The project is motivated by the need for a fast, accessible first-pass interpretation of soil parameters. An interactive classifier can demonstrate how multiple measurements are considered together and can help users explore hypothetical parameter combinations. Comparing a single Decision Tree with a Random Forest also provides a practical way to study the trade-off between interpretability and ensemble performance.

- Provide a quick classification after soil parameters are entered.
- Compare a simple tree model with an ensemble model using common evaluation metrics.
- Show feature-importance plots and confusion matrices to support interpretation.
- Demonstrate a complete applied machine-learning workflow from data preparation to interactive prediction.

1.3 Overview of the Work Carried Out

The supplied folder contains the following main elements: generate_data.py creates a synthetic soil dataset; train_models.py defines preprocessing, training, and evaluation for Decision Tree and Random Forest classifiers; models/ stores serialized estimators, a scaler, a label encoder, and metrics.json; static/plots/ contains

model-comparison, feature-importance, confusion-matrix, and tree-structure graphics; and app.py provides a Streamlit dashboard with live classification, performance analytics, feature/tree insights, and dataset exploration. The folder also includes HTML/CSS/JavaScript and a lightweight server implementation. The presence of a chronic-kidney-disease notebook appears unrelated to the soil-classification workflow and is not used as evidence for this case study.

1.4 Scope and Responsible-Use Statement

The scope is limited to classifying soil quality from six input features and comparing the two included tree-based classifiers. The model output depends on the dataset and labelling procedure used for training. The CSV appears synthetic and the data-generation script assigns classes from a rule-based score with added random variation and percentile thresholds. Therefore, high model performance on this dataset would not by itself establish accuracy on real farm soil samples. Users should confirm measurements through appropriate testing, consider crop and regional context, and consult qualified agricultural professionals before making fertilizer or land-management decisions. The model should not be used as the sole basis for consequential agricultural decisions.

2. PROBLEM STATEMENT

2.1 Case-Study Scenario

A farmer or agricultural learner has a set of soil measurements and wants an initial classification of the soil condition. Traditional laboratory analysis remains important, but it may require time, access, and expert interpretation. The project explores whether a trained classifier can provide a quick preliminary category from the available parameters while exposing model performance and the main variables used by the classifiers.

2.2 Central Case-Study Question

Given pH, nitrogen, phosphorus, potassium, organic-matter percentage, and soil-moisture percentage, which of the two implemented classifiers—Decision Tree or Random Forest—most reliably predicts the dataset's Low, Medium, or High soil-quality label, and how can the prediction be presented transparently to a user?

2.3 Formal Problem Definition

Let each sample be represented by a feature vector x = [pH, N, P, K, organic matter, moisture]. The supervised learning task is to learn a function f(x) that predicts one of three classes: Low, Medium, or High. The project trains two candidate functions, fDT and fRF, and compares them using held-out test metrics and reported five-fold cross-validation. The selected model should be judged not only by accuracy but also by precision, recall, F1-score, confusion-matrix behaviour, and the reliability of the dataset and evaluation procedure.

2.4 Objectives that Guide the Study

- Identify the input features and target labels used by the soil-classification project.
- Prepare feature values for model training using a StandardScaler and encode class labels using a LabelEncoder.
- Train a bounded-depth Decision Tree and a 100-estimator Random Forest, as specified in the training script.
- Evaluate both models using accuracy, macro/weighted precision, recall, F1-score, confusion matrices, and five-fold cross-validation.
- Expose predictions and supporting plots through the included dashboard and model artifacts.
- Document dataset, reproducibility, and responsible-use limitations.

2.5 Assumptions and Limitations

- The six numeric features are assumed to be available and expressed in units consistent with the interface and training data.
- The supplied CSV contains 1,500 rows and the classes High, Medium, and Low; it is generated/synthetic rather than documented as a collection of field-laboratory observations.
- The target column in the supplied CSV is Soil_Type, whereas train_models.py currently attempts to drop Soil_Quality. This naming mismatch prevents the script from training directly on the supplied CSV without reconciliation.
- The metrics file contains saved evaluation results, but because of the target-column mismatch, the report treats them as recorded artifacts rather than claiming that a fresh end-to-end run was completed during this document preparation.
- The saved models and metrics may not correspond to the exact current CSV or current source code version; retraining and versioning are required for reproducibility.
- Feature importance indicates model reliance within this dataset and does not establish causal effects on soil health.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The file data/soil_data.csv contains 1,500 observations and seven columns. Six columns are numeric predictors, and Soil_Type is the class label. The feature ranges in the file-generation script are pH 4.5–8.8, nitrogen 50–480 kg/ha, phosphorus 5–55 kg/ha, potassium 60–320 kg/ha, organic matter 0.5–5.5%, and moisture 5–55%. These are the ranges defined by generate_data.py; actual observed minima and maxima should be rechecked whenever the dataset is regenerated.

Dataset item	Observed / defined detail
Number of rows	1,500
Input features	pH, Nitrogen, Phosphorus, Potassium, Organic_Matter, Moisture
Target column in CSV	Soil_Type
Target classes	High, Medium, Low
Class counts in supplied CSV	High: 501; Low: 500; Medium: 499
Dataset type	Synthetic/generated dataset (per project script)

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) is used to understand the shape and quality of the dataset before training. For this project, EDA should confirm the number of samples and columns, data types, missing values, duplicate rows, class balance, feature ranges, outliers, and relationships between features and labels. It should also verify that the target column is named consistently across the CSV, training script, and dashboard.

3.3 Preparation Steps Actually Performed

The training script shows the intended preprocessing steps: it separates predictors from the target, encodes the target labels with a LabelEncoder, makes an 80/20 stratified train-test split using random_state=42, and fits a StandardScaler on the training features before transforming the training and test sets. Stratification is intended to preserve the class proportions in both subsets. The current script expects a column named Soil_Quality, while the supplied CSV contains Soil_Type; that mismatch must be fixed before this pipeline can be reproduced directly from the provided CSV.

The data-generation script creates 1,500 synthetic samples with a fixed random seed of 42. It computes a rule-based score using thresholds for pH, nutrient levels, organic matter, and moisture, adds random noise, and divides the scores into three classes using percentile thresholds. This provides a controlled demonstration dataset, but its labels are derived from the same features used for prediction.

3.4 EDA Exploration – Implementation

The project includes a Dataset Explorer dashboard tab and plots in static/plots/. The available files support inspecting the dataset and visualizing model behaviour. A complete reproducible EDA record should additionally report missing-value and duplicate checks, summary statistics, histograms or boxplots for each feature, a class-count chart, and a feature-correlation view. No claim is made here that these checks were all executed as part of the current report preparation.

3.5 Summary of EDA Findings

- The supplied CSV has 1,500 rows, six predictor columns, and one categorical target column.
- The class counts are almost balanced: 501 High, 500 Low, and 499 Medium.
- The CSV target name is Soil_Type, which differs from the Soil_Quality name expected by train_models.py.
- The generation logic uses feature thresholds and random noise to construct labels, so learned patterns may reflect those rules rather than all complexities of real soil conditions.
- The dataset and label schema should be validated and versioned before a final reproducible training run.

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Decision Tree Classifier

A Decision Tree classifies an observation by following a sequence of feature-based splits from a root node to a leaf. The supplied training script uses DecisionTreeClassifier with Gini impurity, max_depth=6, min_samples_split=5, and random_state=42. Limiting depth and requiring a minimum number of samples for a split help constrain tree complexity. A single tree is relatively easy to visualize and explain, although it can be sensitive to the training sample and may overfit if not controlled.

4.2 Random Forest Classifier

Random Forest combines predictions from many decision trees trained with sampling and feature subsampling. The supplied script configures 100 estimators, Gini criterion, max_depth=8, max_features='sqrt', and random_state=42. Aggregating trees often reduces the variance of a single tree and can improve generalization, at the cost of a less compact overall explanation. The project includes a feature-importance plot to help interpret which inputs most influence model splits.

4.3 Implementation Workflow

- Read the soil CSV and separate six predictors from the target label.
- Encode the target classes consistently.
- Create a stratified 80% training / 20% testing split with a fixed seed.
- Fit StandardScaler on the training subset and transform training and test features.
- Train the Decision Tree and Random Forest using the parameters described above.
- Generate test-set predictions and compute accuracy, macro and weighted precision/recall/F1, and confusion matrices.
- Run stratified five-fold cross-validation and record mean and standard deviation.
- Save trained models, scaler, label encoder, and metrics as artifacts for the dashboard.

- Use the dashboard to accept six input values and show the predictions of both classifiers.

4.4 Dashboard and Project Components

The Streamlit interface in app.py is organized into four tabs: Live Classification, Performance Analytics, Feature & Tree Insights, and Dataset Explorer. It loads the serialized models, scaler, label encoder, and metrics file; transforms entered parameters; and displays each model's predicted class and probabilities. The repository also contains index.html, app.js, styles.css, and server.py, which provide an additional web interface/server path. These components should be tested as a consistent deployment route and documented separately from the model-training pipeline.

4.5 Reproducibility Note

Before final submission, rename the CSV target column to Soil_Quality or update train_models.py to use Soil_Type, then verify the dashboard and server use the same feature order and class mapping. Retrain the models from the corrected dataset, regenerate metrics and plots, and record the software versions and run commands. This reconciliation is necessary because saved artifacts can otherwise mask a mismatch between the code and data.

5. MODEL EVALUATION AND RESULTS

The values below are transcribed from the supplied models/metrics.json file. They describe the saved evaluation artifact and should be confirmed by rerunning the training pipeline after resolving the target-column mismatch.

Metric	Decision Tree	Random Forest
Test accuracy	63.67%	74.00%
Macro precision	67.86%	73.93%
Macro recall	63.67%	74.00%
Macro F1-score	64.56%	73.96%
Weighted F1-score	64.56%	73.96%
5-fold CV mean accuracy	67.27%	75.73%
5-fold CV standard deviation	3.59 percentage points	4.14 percentage points

Interpretation: in the saved results, Random Forest exceeds Decision Tree by 10.33 percentage points in test accuracy and by approximately 8.40 percentage points in macro F1-score. Its reported cross-validation mean is also higher by 8.47 percentage points. This supports Random Forest as the stronger of the two models in the recorded experiment, while the synthetic nature of the dataset and schema inconsistency limit claims about real-world generalization.

5.1 Feature Importance

The saved metrics assign the largest Decision Tree importance to pH (approximately 0.380), followed by nitrogen (0.216). For Random Forest, pH (approximately 0.277) and nitrogen (0.226) are also the two largest importances, followed by phosphorus. Moisture has a very small Decision Tree importance in the saved artifact (approximately 0.010), while its Random Forest importance is approximately 0.100. These values describe split-based importance in the saved models; they should not be interpreted as proof that one soil factor causes a particular quality outcome.

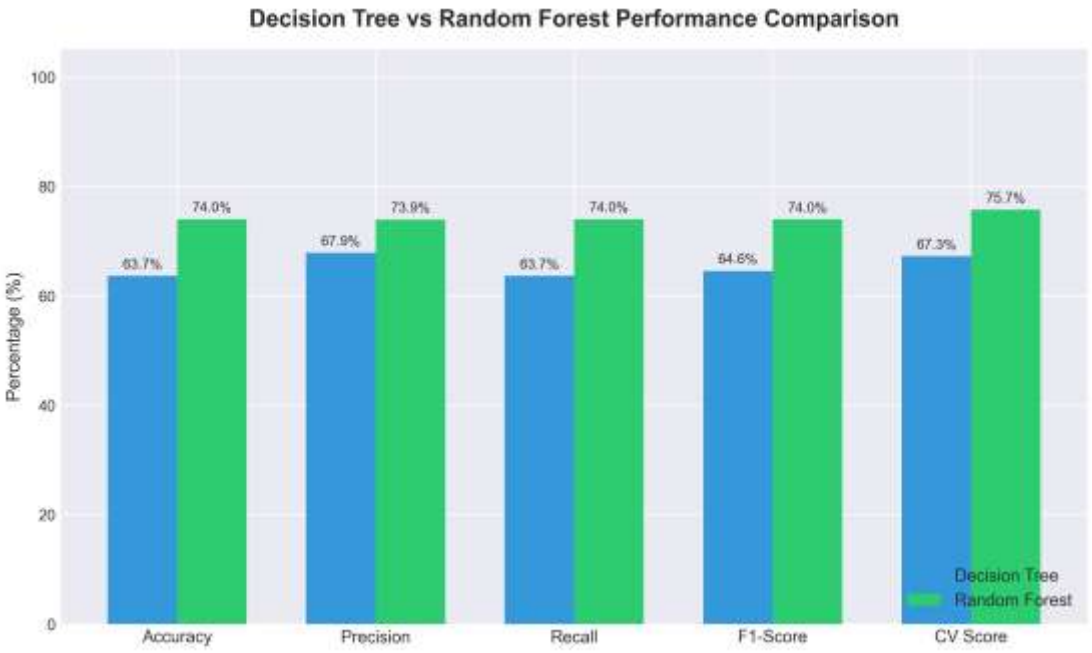


Figure 1. Model comparison plot included in the supplied project folder.

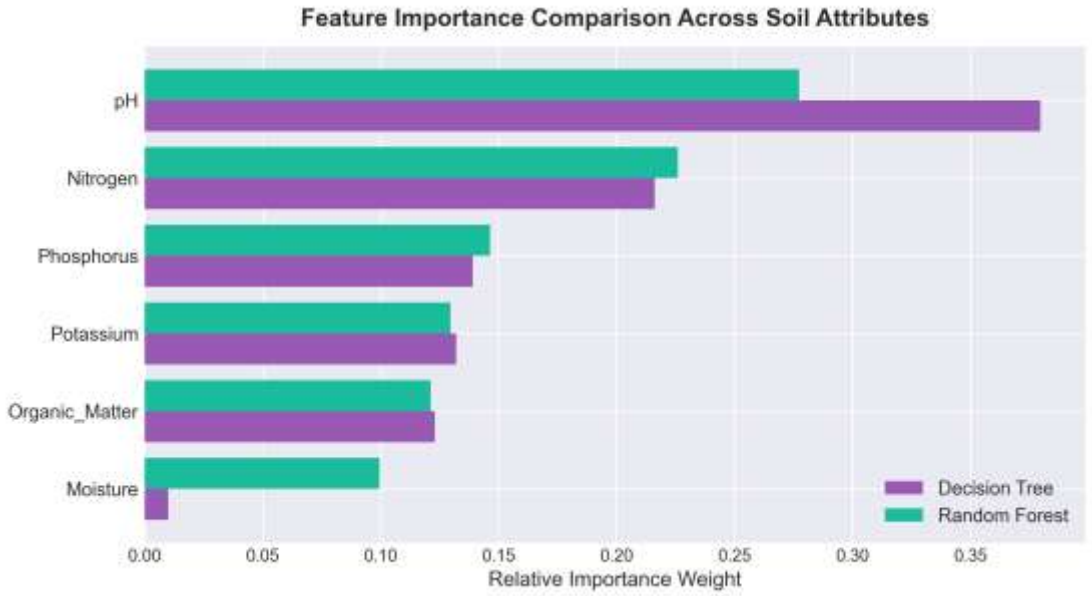


Figure 2. Feature-importance visualization included in the supplied project folder.

6. CONCLUSION

The project demonstrates a supervised machine-learning workflow for classifying soil quality from six soil attributes and comparing a Decision Tree with a Random Forest. According to the supplied metrics artifact, Random Forest provides better test accuracy, macro F1-score, and mean cross-validation accuracy than the single Decision Tree. The dashboard and visualizations make the model comparison more accessible by presenting predictions, performance metrics, and feature-importance information.

The most important next step is to ensure reproducibility: the provided CSV uses Soil_Type, while the current training script expects Soil_Quality. After reconciling this name, the team should rerun training and evaluation,

regenerate the metrics and plots, verify that the dashboard loads the newly generated artifacts, and report the resulting metrics. Since the supplied dataset is synthetic and uses rule-derived labels, validation on suitable real-world soil samples is needed before practical agricultural use.

REFERENCES / PROJECT ARTIFACTS

- Project source files supplied by the team: generate_data.py, train_models.py, app.py, server.py, and requirements.txt.
- Dataset supplied in the project archive: data/soil_data.csv.
- Saved evaluation artifact supplied in the project archive: models/metrics.json.
- Saved visualization artifacts: static/plots/model_comparison.png, feature_importance.png, confusion_matrices.png, and decision_tree_structure.png.
- Project presentation supplied by the team: Soil Quality Classification, Machine Learning (25SC2107E), Team 19, Section 6.

TEAM SIGNATURES

P. Naveen (2520030518): _____P.Naveen_____

J. Advaith (2520030524): _____J.Advaith_____

P. Shashe Preetham (2520030534): _____P.Shashe Preetham_____